

Intro to Object-Oriented Programming (OOP) in Python



In this course, you'll learn how to:

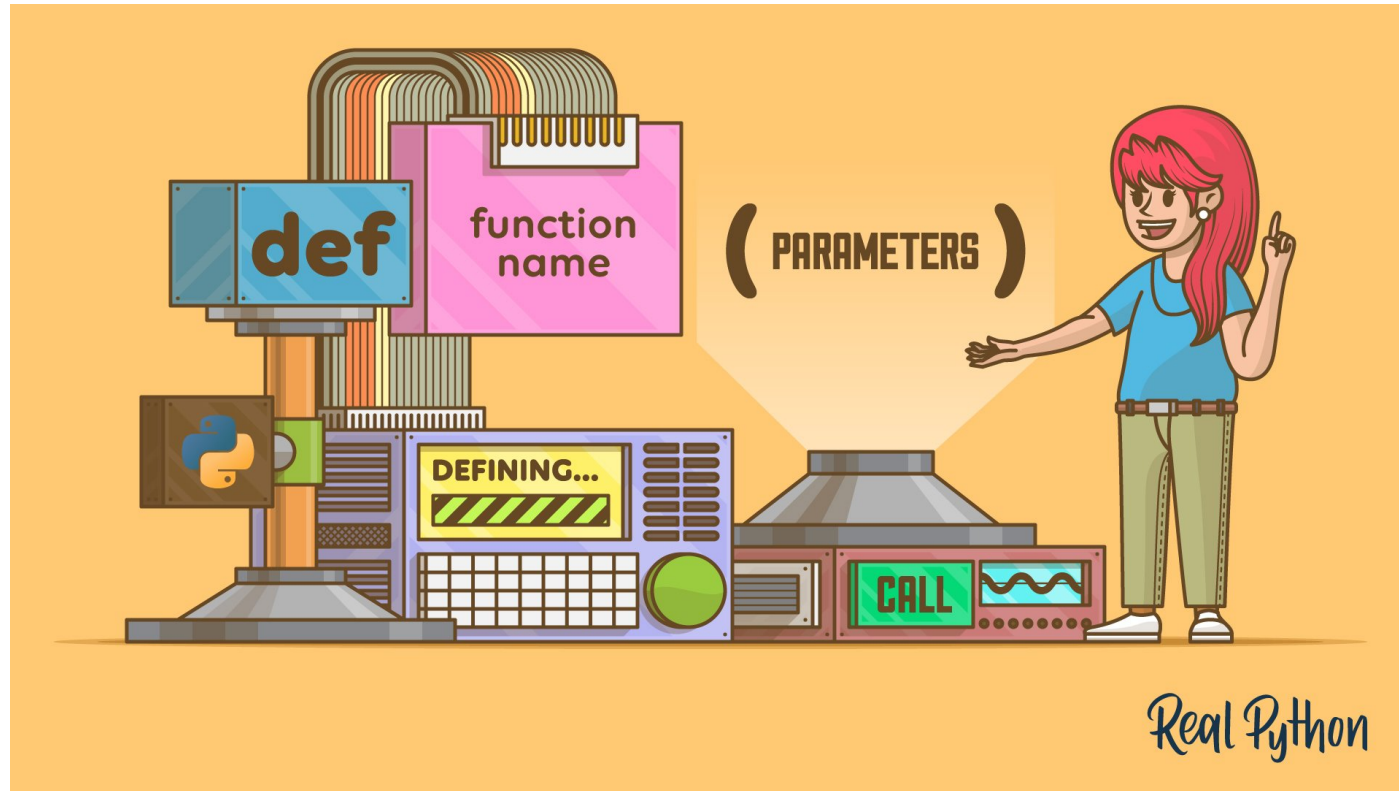
- Use classes to apply object-oriented principles in Python
- Create objects by instantiating classes
- Model properties and behaviors with attributes and methods
- Build child classes using inheritance and `super()`
- Leverage OOP's principles of encapsulation, abstraction, inheritance, and polymorphism to write clean, flexible code

Prerequisites

Python fundamentals:

- Variables
- Functions
- Built-in data types

Defining Your Own Python Function



<https://realpython.com/defining-your-own-python-function/>

Exploring Object-Oriented Programming

Object-oriented programming (OOP) is based on **objects** and **classes**

Objects

- Contain both data and behaviors, called **attributes** and **methods**
- Emulate real-world entities or systems

Classes

- Provide a blueprint for creating objects, called **instances**
- Define the **type** of instances created from the class

Exploring Object-Oriented Programming

Encapsulation

- Bundle data and behaviors
- Promotes integrity and modularity

Inheritance

- Create class hierarchies
- Promotes code reuse and reduce duplication

Abstraction

- Hide implementation details to simplify object interactions
- Focus on what, not how

Polymorphism

- Treat objects of different types as a shared base type
- Code to interfaces, not types

Exploring Object-Oriented Programming

list

```
>>> kirk = ["James Kirk", 34, "Captain", 2265]
>>> kirk[2]
'Captain'
```

- Error prone
- Hard to maintain
- Difficult to scale

Crew class

```
>>> kirk = Crew("James Kirk", 34, "Captain", 2265)
>>> kirk.rank
'Captain'
```

- Consistent interface
- Access attributes by name
- Extendable

Defining Classes in Python

Classes are the blueprints for objects in Python

```
class Crew:  
    pass
```

- Define a class with the `class` keyword
- Indent the contents of the class four spaces
- **Instantiate** the class by calling it with parentheses
 - `crew_instance = Crew()`

Working With Attributes

In OOP in Python, objects can have two kinds of attributes:

Class attributes

- Defined directly in the class body
- Shared by all instances of the class

Instance attributes

- Assigned when an instance is **initialized**
- Unique to each instance

Working With Attributes

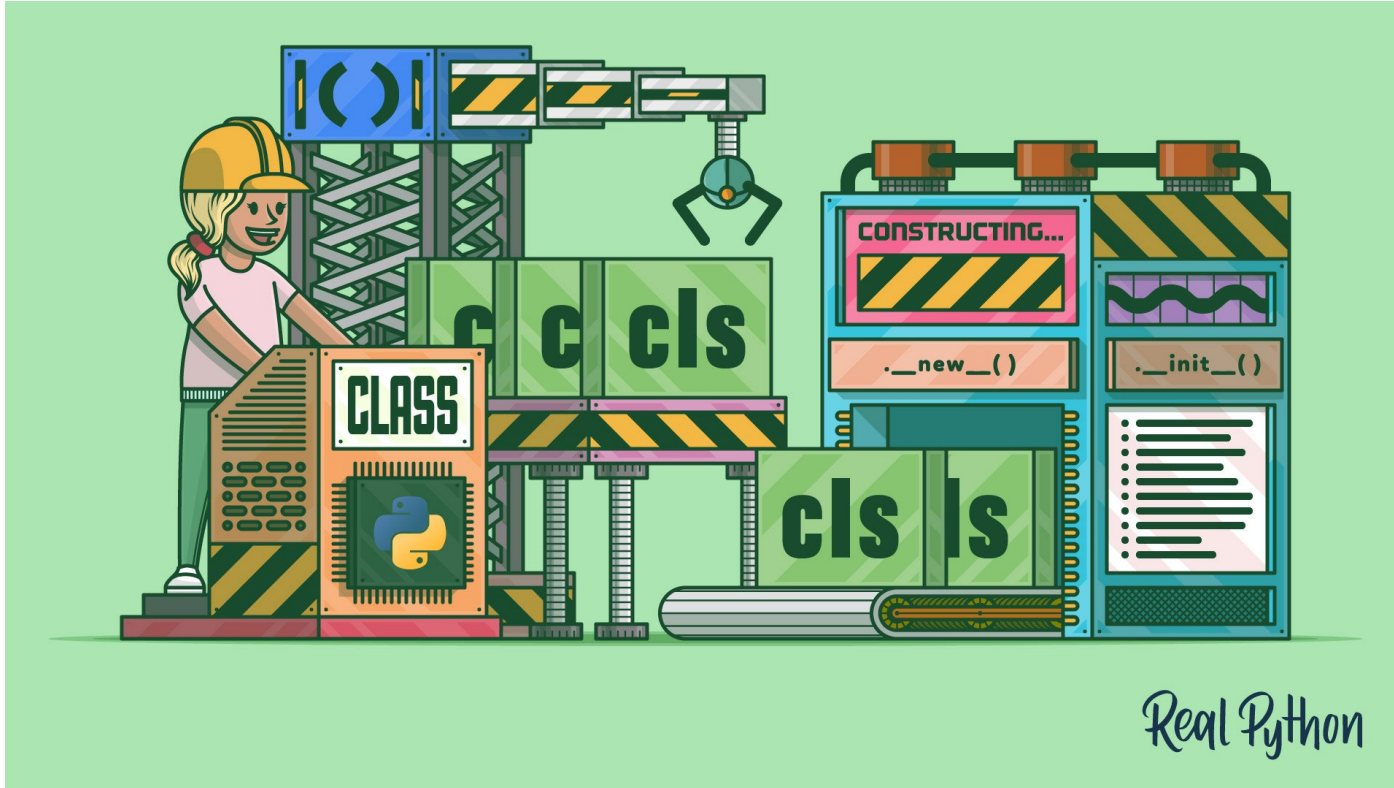
`__init__()` is a special method that runs on object instantiation

```
class Crew:
    species = "Human"

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

- `species` is a class attribute, available on all instances
- `self` takes on the value of the newly created object
- `name` and `age` are assigned to `self` as instance attributes

Python Class Constructors: Control Your Object Instantiation



<https://realpython.com/python-class-constructor/>

Giving Your Classes Behavior With Methods

Methods are functions defined within a class

Instance methods

- Define behaviors for class instances
- Can be called on class instances using the dot accessor
- Provide encapsulation by bundling functionality with data

Giving Your Classes Behavior With Methods

Instance methods are defined in the class body with the `def` keyword

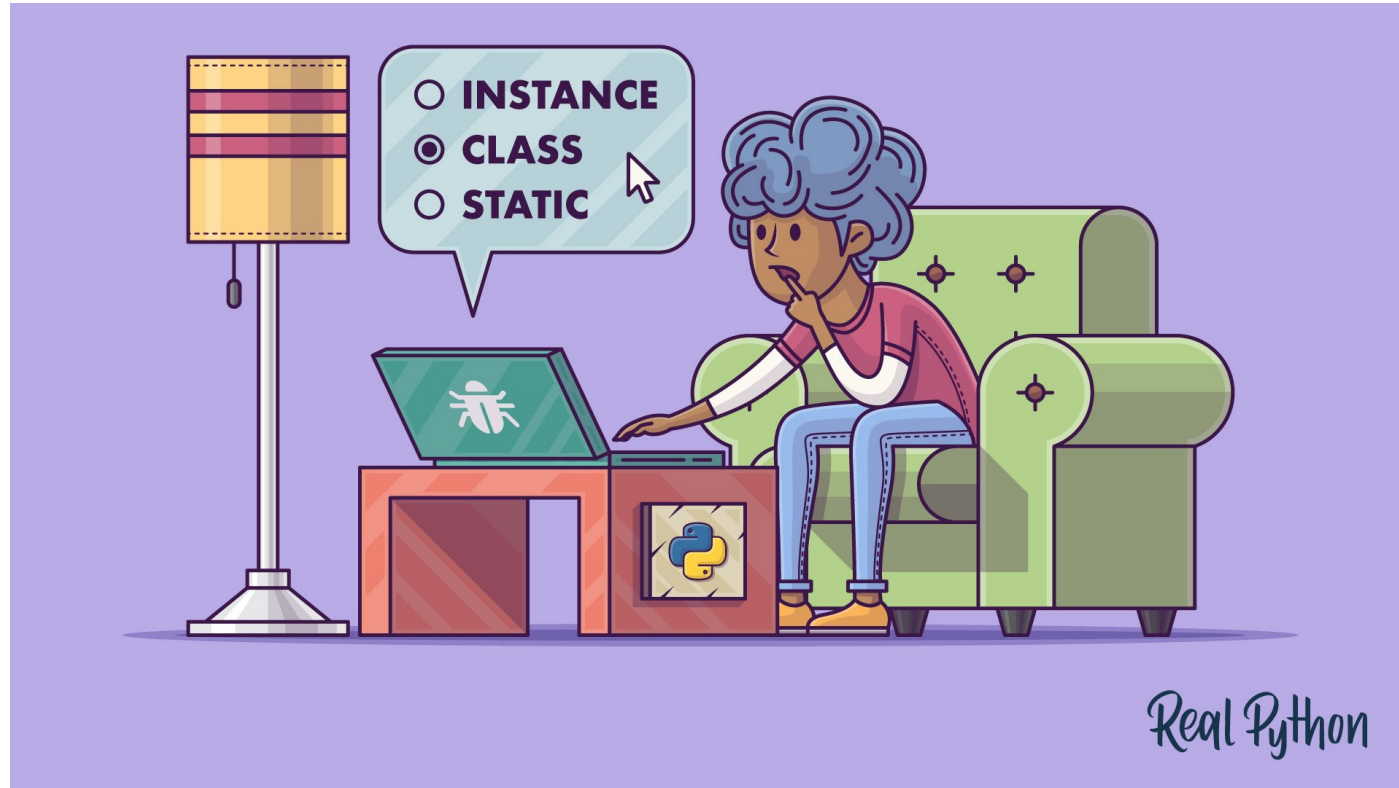
```
class Crew:

    ...

    def description(self):
        return f"{self.name} is {self.age} years old"
```

- Can be called on class instances using the dot accessor
 - `crew_instance.description()`
- `self` takes on the value of instance calling the function

Python's Instance, Class, and Static Methods Demystified



<https://realpython.com/instance-class-and-static-methods-demystified/>

Understanding Inheritance in Python

Inheritance

- Lets one class acquire the attributes and methods of another
- Defines the newly created class as a child class, or subclass
- Defines the original class as the parent, or base class
- Enables you to extend or repurpose existing classes

Understanding Inheritance in Python

With inheritance, attributes and methods from the parent class can be:

Inherited directly

- Parent behavior or data is used as-is

Overridden

- Parent behavior or data is replaced

Extended

- Parent behavior or data is augmented or modified

Understanding Inheritance in Python

```
class Parent:  
    hair_color = "brown"  
  
class Child(Parent):  
    pass
```

- The parent class is passed in the child class definition: `Child(Parent)`
- `Child` inherits all of the attributes and methods in `Parent`
- The `.hair_color` attribute on `Parent` and `Child` instances is `"brown"`

Understanding Inheritance in Python

```
class Parent:  
    hair_color = "brown"  
  
class Child(Parent):  
    hair_color = "purple"
```

- `Child` overrides the `.hair_color` attribute defined in `Parent`
- The `.hair_color` attribute on only `Child` instances is `"purple"`

Understanding Inheritance in Python

`super()` lets a class call parent methods and access parent attributes

```
class Parent:
    def __init__(self):
        self.speaks = ["English"]

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.speaks.append("German")
```

- `super().__init__()` calls the `__init__()` of the `Parent` class
- The `.speaks` attribute of `Child` instances is `["English", "German"]`

Intro to Object-Oriented Programming (OOP) in Python



Summary

In this course, you learned:

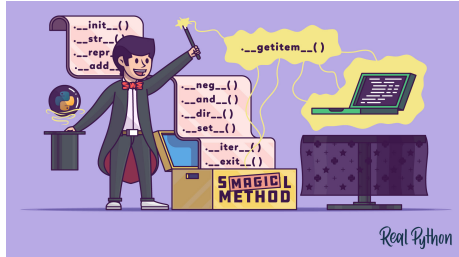
- Object-oriented programming in Python uses classes as blueprints for objects
- You can create objects by instantiating classes
- Attributes and methods define the properties and behaviors of objects
- Child classes can use `super()` to invoke parent class methods
- How to use the four pillars of object-oriented programming to write clean, maintainable programs

Object-Oriented Programming (OOP) in Python

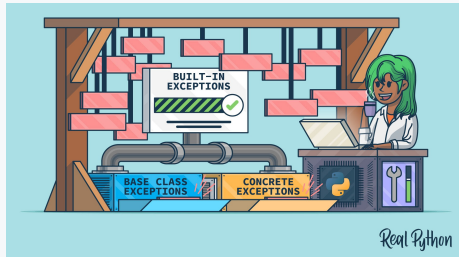


<https://realpython.com/python3-object-oriented-programming/>

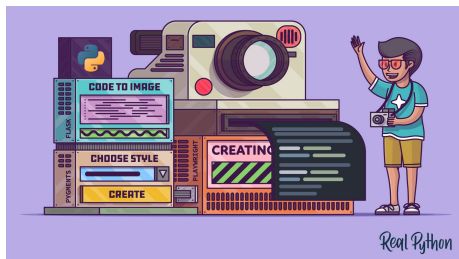
Additional Resources:



Python's Magic Methods in Classes



Working With Python's Built-in Exceptions



Building a Code Image Generator With Python

👋 **Thank You!** 👋